

Traffic Crash Analysis

Author

Saurav Upadhyaya

Table of Contents

1. Introduction	3
1.1. Motivation	3
1.2. Project Description	3
2. System Design	4
2.1. Chosen Big Data Frameworks	5
2.2 . Selection of Datasets	5
3. Detailed Description of Components	6
3.1. Important Components	6
3.2. Other components	6
3.3. Detailed Description of Each Component	6
4. Evaluation and Test	7
4.1 Software Manual	7
4.1.1. Evaluation Setup	7
4.1.2 Application Execution	8
Scenario 1: Steps for Confirming the Workers' Execution	9
Scenario 2: Steps for Visualizing Crash Analysis	10
4. 2 Evaluation/Test Environment	11
4.2.1 Docker Configuration:	11
4.2.2. Spark Cluster Configuration	12
4.2.3 Evaluation/Test Results	12
4.2.4 Summary of Results	15
5. Conclusion	16
Reference	17

1. Introduction

Chicago is considered one of the congested cities in the United States, with a population of around 3 million (2024). Every year, it is estimated that 100,000 vehicles are involved in accidents. Thus, it is necessary to analyze the crash data to understand the crash trends and patterns, aiming to provide suggestions and recommendations to the Department of Public Safety authorities. In this project, crash-relevant datasets (~ 2GB), such as crashes, people, vehicles, and Vision Zero traffic fatalities owned by the Chicago Police Department (Data Owner), are used to create geo-spatial visualizations.

1.1. Motivation

Traffic crashes are a major urban safety issue in places like Chicago, causing injuries, fatalities, and property loss each year. Because crashes occur under many different circumstances, I need to identify which factors increase the risk of fatal outcomes. The city collects extensive crash records, but the data are complex and hard to analyze without appropriate tools. In this project, I used PySpark to transform raw crash records into actionable insights. For instance, by combining datasets such as traffic crashes, traffic fatalities, vehicles, and people involved in crashes and using information such as driver's age, crash location, driver's license issuing state, and roadway surface condition, I aim to identify trends, patterns, and critical factors that lead to fatal outcomes.

1.2. Project Description

In this project, I used crash-relevant datasets (~ 2GB) such as crashes, people, vehicles, and Vision Zero traffic fatalities owned by the Chicago Police Department (Data Owner). For processing big data, I used PySpark to process and analyze traffic crash data. In my application, I have used the “apache-spark” image ([link](#)) for managing one master and two workers. This is done to simplify dependency management and ensure a consistent runtime environment across all nodes. 1 Spark Master and 2 Spark Workers are the core components in my Docker containers.

2. System Design

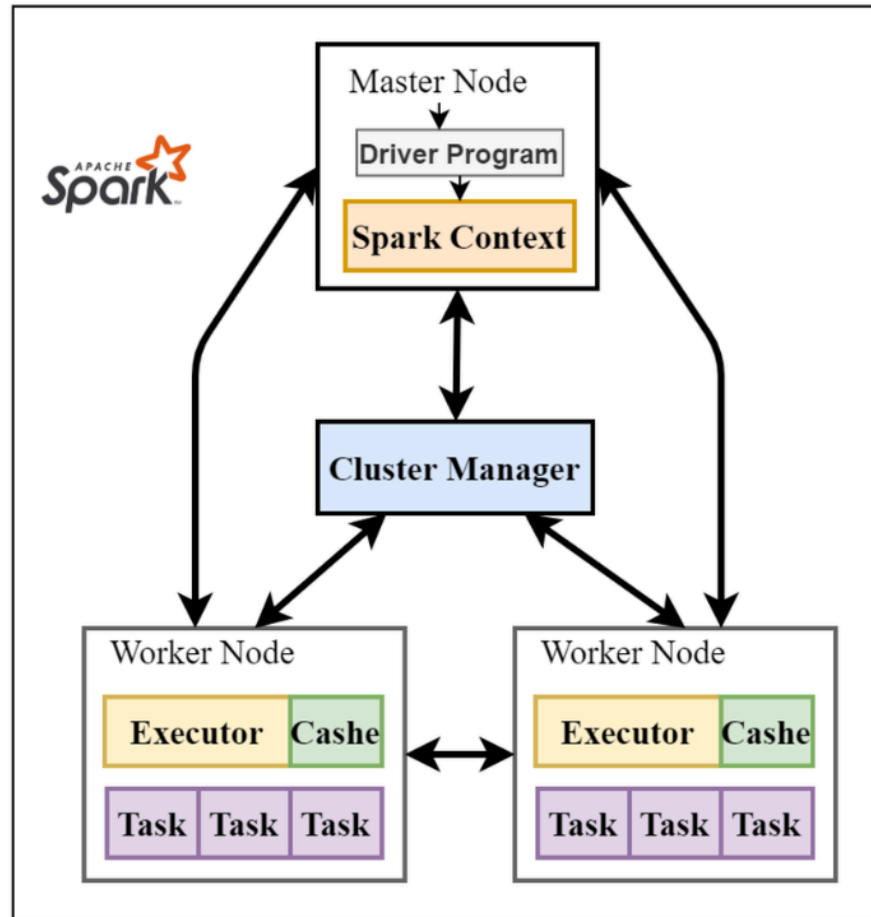


Fig. 1. Overview of Spark Framework

Figure 1 represents the overview of Spark Framework [1]. In my application, I have used the “apache-spark” image ([link](#)) for managing one master and two workers. This is done to simplify dependency management and ensure a consistent runtime environment across all nodes. 1 Spark Master and 2 Spark Workers are the core components in my Docker containers. In my project, I have created a docker-compose.yaml, which includes three services (one spark-master, one spark-worker-1, and one spark-worker-2). In each of these services, I have included several properties, such as “build” to reference the Dockerfile and image for referencing the “apache-spark” image. In each service, I have also used several other properties, such as container name, host name, ports, volumes (for mounting the host directory), and a default command that runs once the container is started from the image.

In my application, the Spark master container distributes the job to workers (Spark worker containers) based on their availability. When I run the spark-submit, the “spark-master” service is triggered, which schedules the tasks that need to be executed by the executors running

within the worker containers, making it easier to replicate my environment for development and testing.

2.1. Chosen Big Data Frameworks

In this project, I have selected “Apache Spark” as a Big Data framework because of its faster execution ability. Thus, I have selected it for big data processing and analytics. I have written all Python Scripts using PySpark.

2.2 . Selection of Datasets

SN	Dataset name	URL
1.	Traffic crashes - crashes	https://data.cityofchicago.org/Transportation/Traffic-Crashes-Crashes/85ca-t3if/about_data
2.	Traffic crashes - vehicles	https://data.cityofchicago.org/Transportation/Traffic-Crashes-Vehicles/68nd-jvt3/about_data
3.	Traffic crashes - people	https://data.cityofchicago.org/Transportation/Traffic-Crashes-People/u6pd-qa9d/about_data
4.	Traffic crashes - vision zero Chicago traffic fatalities	https://data.cityofchicago.org/Transportation/Traffic-Crashes-Vision-Zero-Chicago-Traffic-Fatali/gzaz-isa6/about_data

Figure 2: Dataset Information

In the traffic crashes dataset, I have used traffic crashes datasets: crashes, vehicles, people, and vision zero fatalities (fatalities). I explored the schema and understood the features of each dataset while working on my tasks. I understood that crash data includes information about crashes happening in every city in Chicago. I found that the vehicles dataset includes some information about the vehicles involved in a traffic crash. Likewise, I noticed that the details of each victim are included in the people's dataset. In the same way, I learned that the fatalities dataset information includes important data/information that helps to reform the new policies for minimizing the traffic crashes in Chicago.

3. Detailed Description of Components

3.1. Important Components

1. Master Node
2. Cluster Manager
3. Worker Nodes

3.2. Other components

1. Docker Image
2. Docker Desktop Application
3. Virtual Machines
 - 1 master (spark-master): 6 GB
 - 2 workers (spark-worker-1 and spark-worker-2): 6GB each
4. Spark Driver
5. Vim
6. Visual Studio Code and Colab

3.3. Detailed Description of Each Component

1. **Master Node:** I have used the Master Node, which assigns the workload to workers based on their capacity and availability. In my project, it hosts the Spark Master process.
2. **Cluster Manager:** In my project, when I use the “spark-submit” function, Cluster Manager receives the job request, determines the available worker, and allocates the required resources for executing the job.
3. **Worker Nodes:** I have created separate Docker containers that host a separate Spark process. Once I use the “spark-submit” function, as they have a connection with the Master, they start executing based on their availability and resources.
4. **Docker Image:** I have used the “apache-spark” image for developing a Spark-based application.
5. **Docker Desktop Application:** I have used it for managing my “spark-cluster” named container. It starts the “spark-cluster” named container (in my case), which automatically runs the master and 2 workers.
6. **Virtual Machines (VMs):** I have created a Master and two workers. Master controls the workflow execution and assigns work to each worker (spark-worker-1 and spark-worker-2). In fact, when a Master is assigned the job, it is distributed to workers based on their capacity and workload.
7. **Spark Driver:** Spark Driver is the brain of the Spark application and makes the resources available for the next job. When I use the “spark-submit” function in a terminal for submitting my application, Spark automatically distributes the code written in a Python file and dependencies to the cluster. Then, the driver communicates with the Master,

which then distributes the job to its workers. Once the job is done, the driver gets the result of that Python file.

8. **Vim:** I have used it for writing scripts and creating Python files.
9. **Visual Studio Code:** I have used it (for executing Spark locally), and Colab for data collection, cleaning, integration, analysis, and visualization.

4. Evaluation and Test

4.1 Software Manual

4.1.1. Evaluation Setup

- 1) Installation of Docker, Docker Desktop, Python, and Vim in the terminal
- 2) Run the Docker Daemon (from Docker Desktop application)
- 3) Extract the uploaded Zip folder
- 4) Navigate to the project directory (eg, /Users/[ymy-user-name]/spark-cluster)
- 5) From the project directory (spark-cluster), use the command below to transfer the data to the Master VM.

```
~/spark-cluster: docker cp traffic_crashes_data spark-master:/traffic_crashes_data
```

- 6) Again, from the project directory (spark-cluster), use the command below;

```
~/spark-cluster: docker cp data_access_confirmation.py  
spark-master:/data_access_confirmation.py
```

- 7) Use the two scripts shared below to launch the Spark Cluster on the local machine

Script:

```
—  
docker-compose build --no-cache  
docker-compose up  
—
```

Note: I have defined three services such as one for the master and the other two for workers, which contain different properties, such as container name, host name, ports, volumes (for mounting the host directory), and a default command that runs once the container is started from the image. So, when I use the above script, it starts all those services.

- 8) Now, verify the status of the containers using the command below.

Command: `docker ps`

Note: I can also see the connected workers list in Spark UI at <http://localhost:8080/>

Reference:

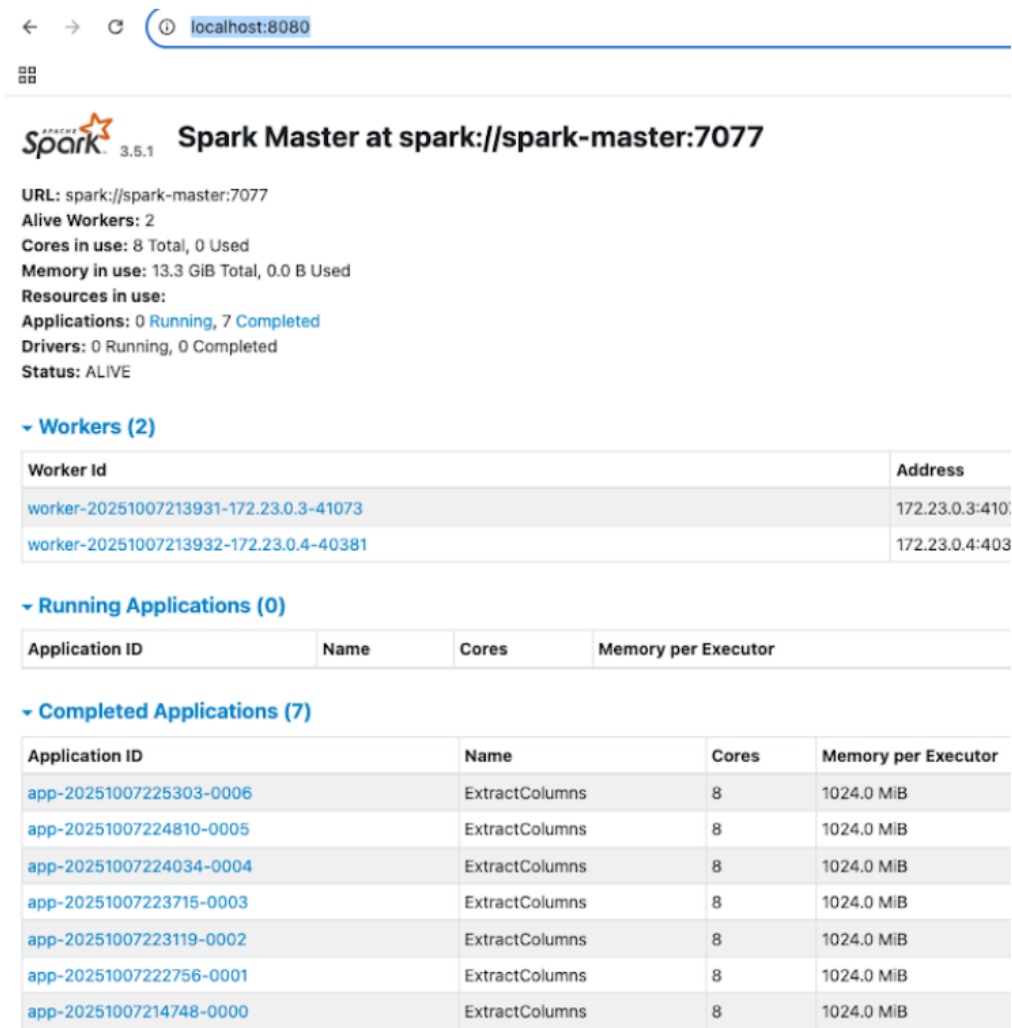


Figure 3: Spark UI

4.1.2 Application Execution

- 1) From the project directory (spark-cluster), use the command below.

```
~/spark-cluster: docker cp get_column_names_from_all_csv_files.py  
spark-master:/get_column_names_from_all_csv_files.py
```

Note: This command is executed so that I can transfer my project file to the master

```
~/spark-cluster: docker cp data_access_confirmation.py  
spark-master:/data_access_confirmation.py
```

- 2) Now, execute the command below to enter the Master VM

Command: `docker exec -it --user root spark-master bash`, and I will enter as shown in the reference script below.

Reference:

from: `~/spark-cluster: docker exec -it --user root spark-master bash`

to: `root@spark-master:/app#`:

- 3) From the Master VM directory, execute the Python script using the command below.
`root@spark-master:/app# /opt/spark/bin/spark-submit --master spark://spark-master:7077 /get_column_names_from_all_csv_files.py`
 Note: Once this command is executed in the Master VM, it distributes the workload to available workers and provides the resources as required.

Scenario 1: Steps for Confirming the Workers' Execution

- a) Goal: To get the list of all columns present in my datasets, such as crashes, people, vehicles, and fatalities.
- b) Datasets: Refer to section 2.2 to get all chosen datasets
- c) Select the project file named `get_column_names_from_all_csv_files.py` from the uncompressed zip folder
- d) Use the three commands below;
 1. `~/spark-cluster: docker cp get_column_names_from_all_csv_files.py spark-master:/get_column_names_from_all_csv_files.py`
 2. `~/spark-cluster: docker exec -it --user root spark-master bash`
 3. `root@spark-master:/app# /opt/spark/bin/spark-submit --master spark://spark-master:7077 /get_column_names_from_all_csv_files.py`
- e) Outcome: I saw that Master VM can distribute workload to its workers. I have two workers, and they executed the tasks.

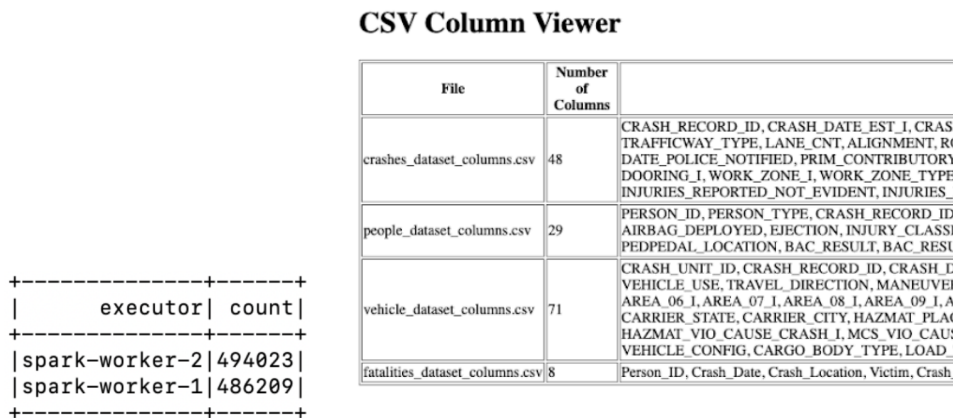


Figure 4: Worker Execution (a) and List of all Columns (b)

Scenario 2: Steps for Visualizing Crash Analysis

- a) Goal: To visualize the Daily Crashes vs the Monthly Crashes in Chicago
- b) Datasets: Refer to section 2.2 to get the crashes dataset, and select the “CRASH_DATE” feature
- c) Select the project file named `cleaning_data_keys_parsing_fields.py` from the uncompressed zip folder
- d) Use the three commands below;
 1. `~/spark-cluster: docker cp cleaning_data_keys_parsing_fields.py spark-master:/cleaning_data_keys_parsing_fields.py`
 2. `~/spark-cluster: docker exec -it --user root spark-master bash`
 3. `root@spark-master:/app# /opt/spark/bin/spark-submit --master spark://spark-master:7077 /cleaning_data_keys_parsing_fields.py`
- e) Now again, from the same project directory where Docker is initiated, transfer the image to the local machine.
`~/spark-cluster docker cp spark-master:/app/Daily_Monthly_Crashes.png ~/Desktop/crash_analysis`
- f) Outcome: I get the `crash_analysis.png` image file that shows the visualization as shown below.

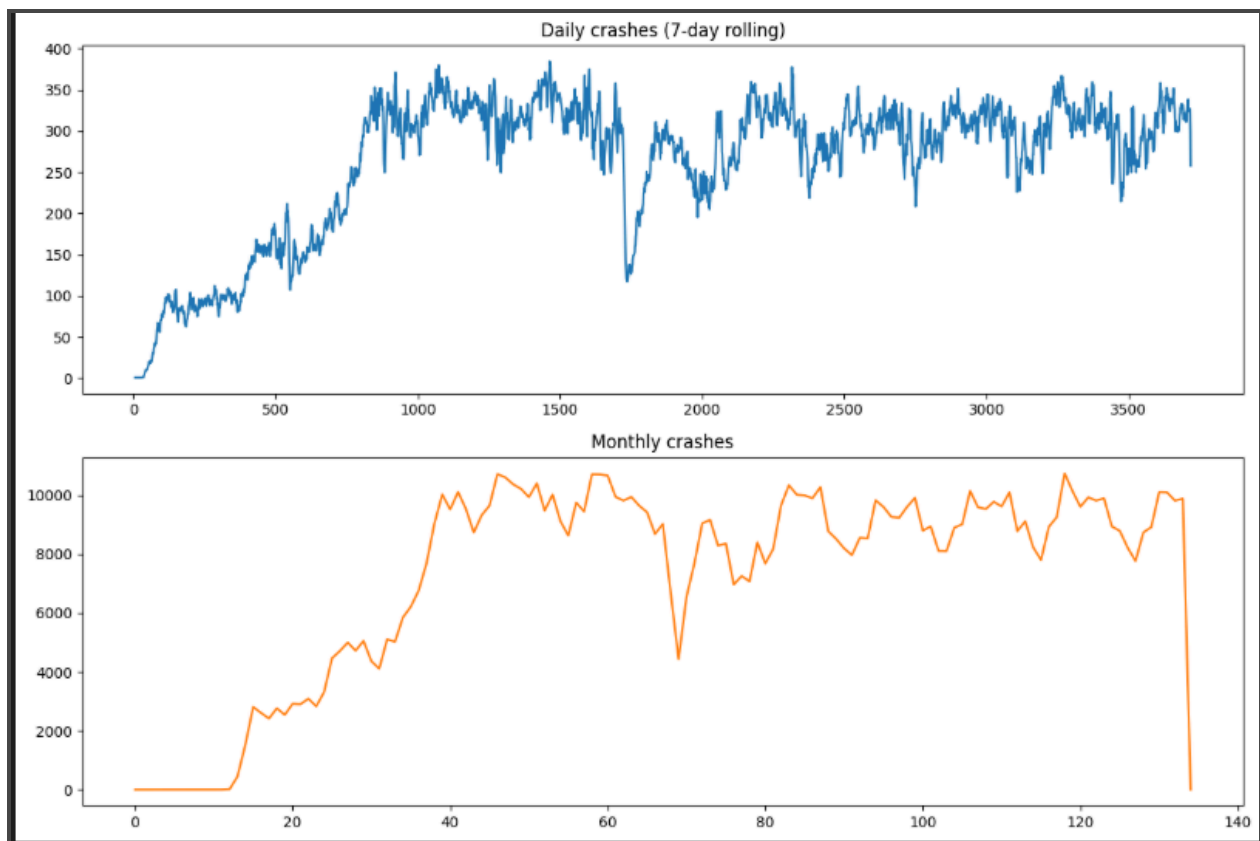


Figure 5: Daily Crashes vs Monthly Crashes

4. 2 Evaluation/Test Environment

4.2.1 Docker Configuration:

Docker compose file (docker-compose.yaml)

```
version: '3.8'

services:
  spark-master:
    build:
      context: .
      dockerfile: Dockerfile
    image: apache/spark-py
    #image: my-pyspark-with-tools:latest
    container_name: spark-master
    hostname: spark-master
    ports:
      - "7077:7077"
      - "8080:8080"
    volumes:
      - /Users/sauravupadhyaya/desktop/fall2025/big_data_course/final_project/traffic_crashes_data:/data
      # - /Users/sauravupadhyaya/desktop/big_data_project/requirements.txt:/opt/requirements.txt
      - ../big_data_project:/tmp/spark/work-dir/big_data_project
      - ./app
    command: >
      bash -c "/opt/spark/bin/spark-class org.apache.spark.deploy.master.Master --host spark-master"

  spark-worker-1:
    build:
      context: .
      dockerfile: Dockerfile
    image: apache/spark-py
    #image: my-pyspark-with-tools:latest
    container_name: spark-worker-1
    hostname: spark-worker-1
    depends_on:
      - spark-master
    ports:
      - "8081:8081"
    environment:
      - SPARK_WORKER_DIR=/tmp/spark/work
    volumes:
      - /Users/sauravupadhyaya/desktop/fall2025/big_data_course/final_project/traffic_crashes_data:/data
      - /tmp/spark/work:/tmp/spark/work
      # - /Users/sauravupadhyaya/desktop/big_data_project/requirements.txt:/opt/requirements.txt
      - ./app
    command: >
      bash -c "/opt/spark/bin/spark-class org.apache.spark.deploy.worker.Worker spark://spark-master:7077"
      - ../big_data_project:/tmp/spark/work-dir/big_data_project

  spark-worker-2:
    build:
      context: .
      dockerfile: Dockerfile
    image: apache/spark-py
    #image: my-pyspark-with-tools:latest
    container_name: spark-worker-2
    hostname: spark-worker-2
    depends_on:
      - spark-master
    environment:
      - SPARK_WORKER_DIR=/tmp/spark/work
    volumes:
      - /Users/sauravupadhyaya/desktop/fall2025/big_data_course/final_project/traffic_crashes_data:/data
      - /tmp/spark/work:/tmp/spark/work
```

Figure 6: Docker compose file structure

In my project, I have created a docker-compose.yaml, which includes three services (one spark-master, one spark-worker-1, and one spark-worker-2. In each of these services, I have

included several properties, such as “build” to reference the Dockerfile and image for referencing the “apache-spark” image. In each service, I have also used several other properties, such as container name, host name, ports, volumes (for mounting the host directory), and a default command that runs once the container is started from the image. In my project, containers communicate through an internal Docker network.

4.2.2. Spark Cluster Configuration

- a) Cluster mode: Spark Standalone mode
- b) Nodes: 1 Master and 2 Workers

4.2.3 Evaluation/Test Results

- a) In my application, I used Colab and Visual Studio Code for using a single machine (local [*]), and I compared the time it takes to execute the `get_column_names_from_all_csv_files.py` Python file. I observed that my cluster (1 Master and 2 Workers) executes very fast compared to a single machine.
- b) In the SPARK UI, I also observed that workers were assigned the job based on their availability and capacity.

Objective: Geospatial Analysis of Fatal Crash Characteristics

Some major geospatial visualizations obtained from data analysis using PySpark in Chicago Traffic Crashes (crashes, people, vehicles, and zero vision fatalities) datasets:

Example 1: Total number of crashes happening in each city ([link](#))

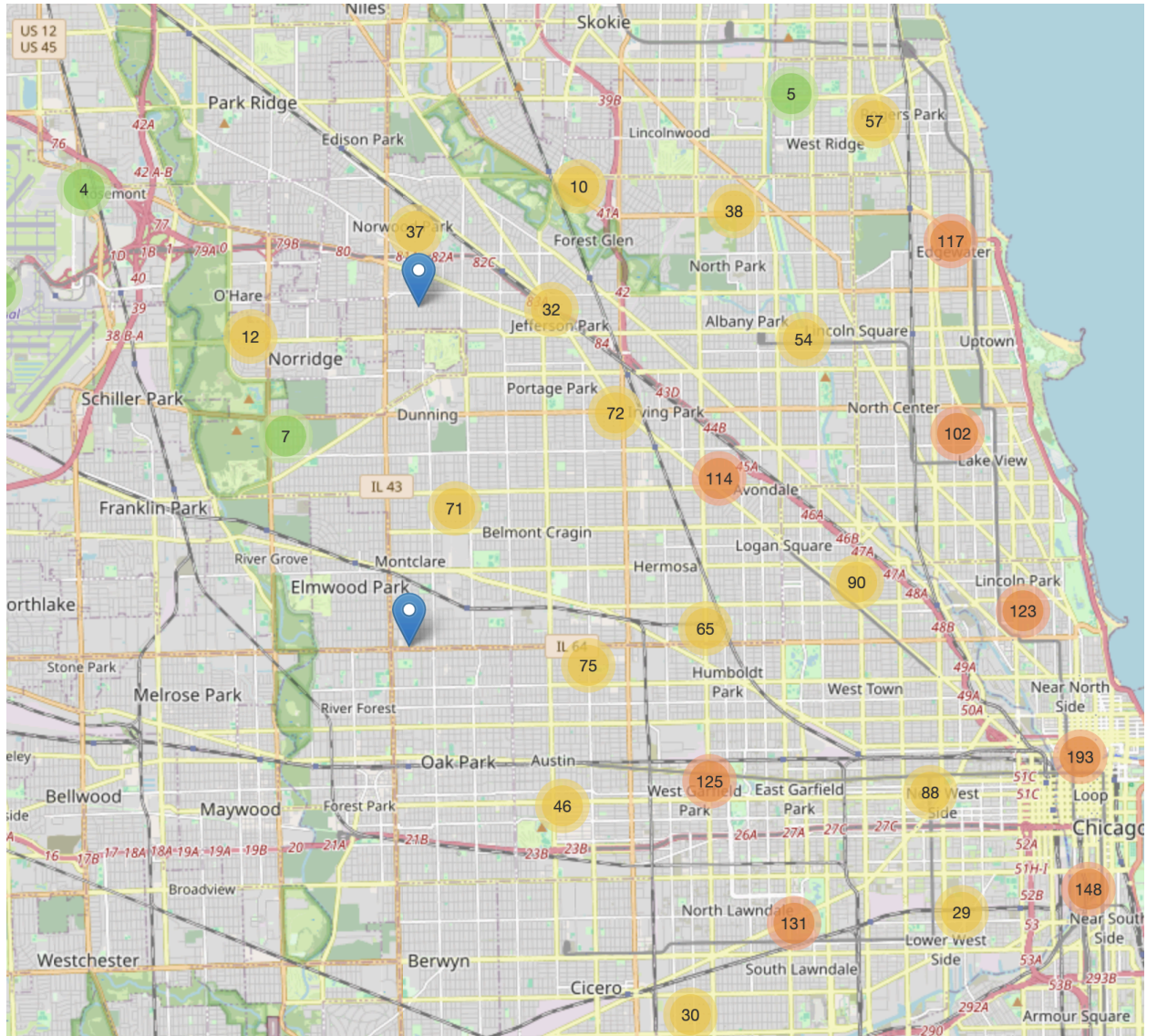


Figure 7. Total number of crashes happening in each city

In the above figure, I see the number of crashes happening per city in Chicago.

Example 2: Fatal Crashes: Cyclists ([link](#))

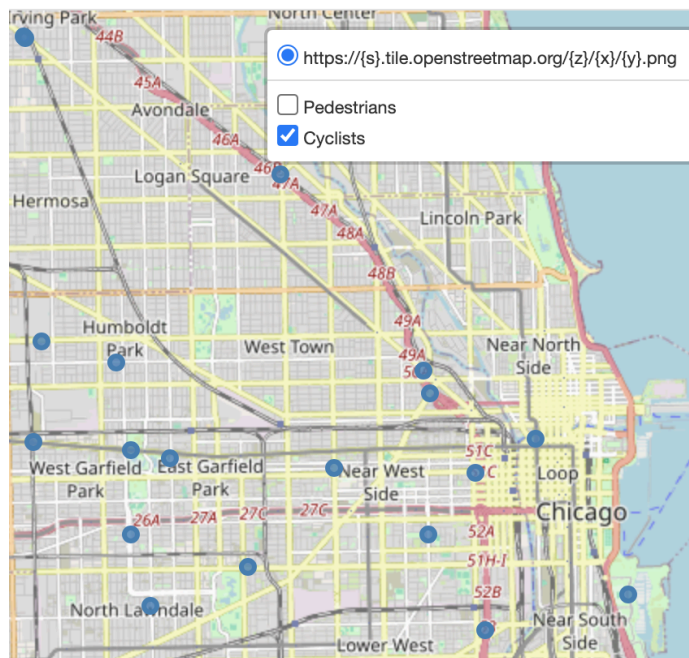
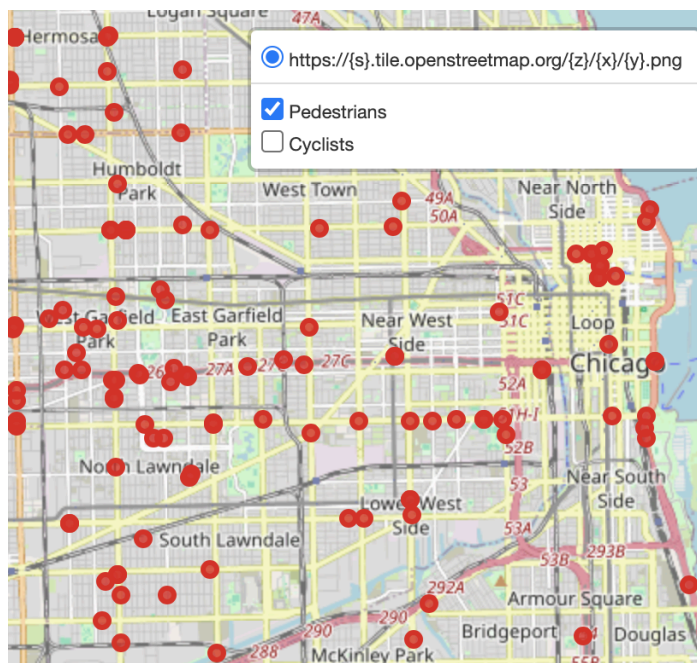


Figure 8. Fatal crashes: Cyclists

In the above figure, I see the number of blue markers representing the death of the Cyclists.

Example 3: Fatal Crashes: Pedestrians ([link](#))



Example 4: Fatal Traffic Accidents Analysis ([link](#))

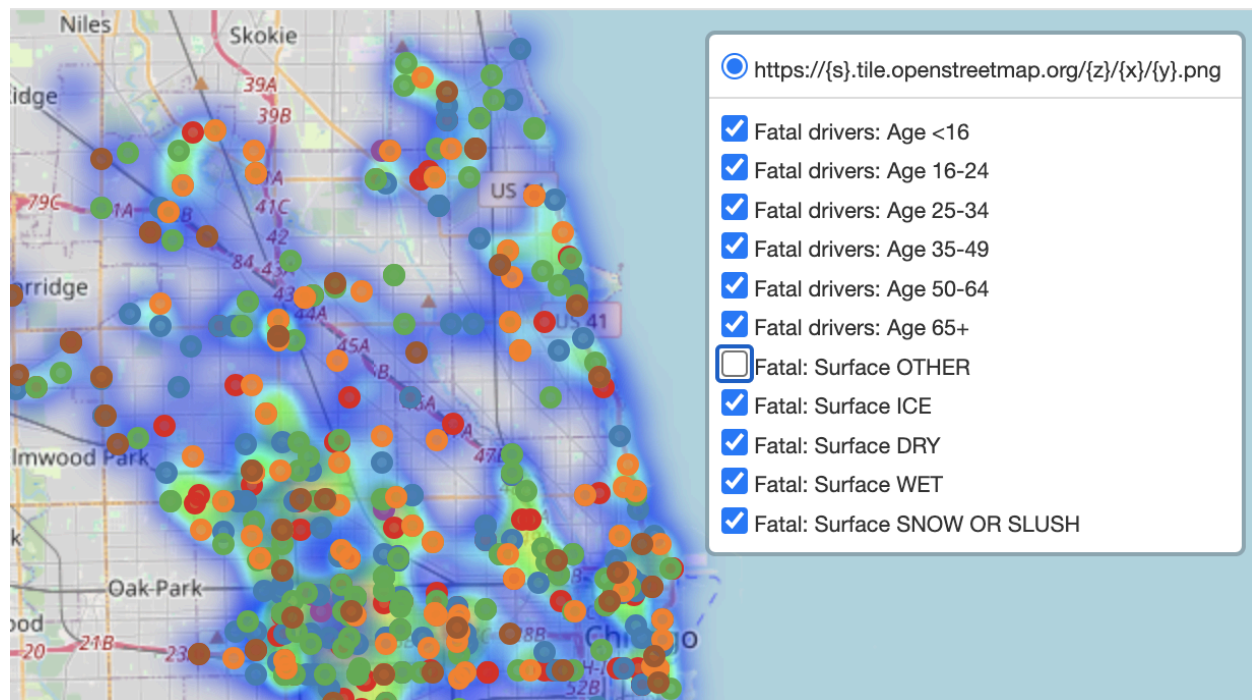


Figure 10. Fatal Traffic Accidents Analysis

In the above figure, I see that the fatal crashes are clustered in certain areas in which age groups and road conditions are key factors.

Some other geospatial visualizations obtained from data analysis using PySpark:

- Crashes (top 5 causes): ([link](#))
- Visualizing Crashes, Injury, and Fatal: ([link](#))
- Crashes (Heatmap): ([link](#))
- Risk analysis: ([link](#))
- Vehicles' fatal heavy hit run visualization: ([link](#))

4.2.4 Summary of Results

In this project, by creating the “Total number of crashes happening in each city” geospatial visualization ([link](#)), I found the number of crashes happening in each city of Chicago. Likewise, by creating “Fatal Crashes: Cyclists and Pedestrians” ([link](#)) visualization, I found the cities representing the deaths of cyclists and pedestrians. In the same way, by creating the “Fatal Traffic Accidents Analysis” ([link](#)) visualization, I see that the fatal crashes are clustered in certain areas in which age groups and road conditions are key factors.

5. Conclusion

In this project, I used PySpark on the Traffic Crashes dataset (2 GB) for distributed data processing and geo-spatial visualizations. Likewise, I used an Apache Spark Docker image to create one VM and two Worker VMs (each of 6 GB). When Master gets the task, it distributes the job to its workers based on their availability. On analyzing the crash dataset, I observed that 100k+ crashes are happening every year in Chicago. By creating the crash markers geospatial visualization, I found the number of crashes happening per city in Chicago. Likewise, by creating a fatal geo-spatial visualization, I can analyze the geographic locations of the fatal car crashes by examining two important factors, such as the age group of the driver and the road surface conditions at the time of the crash. Similarly, by creating a fatal heavy hit and run geo-spatial visualization, I found the hotspots where the fatal crashes happen frequently. Thus, these visualizations provide recommendations (eg, policy making) to improve road safety. In the future, I am planning to use ML models to forecast the crash-prone areas and analyze the real-time data of other states as well.

Reference

1. Ullah, Rahmat & Arslan, Tughrul. (2020). PySpark-Based Optimization of Microwave Image Reconstruction Algorithm for Head Imaging Big Data on High-Performance Computing and Google Cloud Platform. *Applied Sciences*. 10. 3382. 10.3390/app10103382.